

I design and implement compiler/runtime and backend systems where correctness depends on explicit ownership, state, and lifecycle boundaries. UniversalToolchain resolves composition into one immutable LanguagePlan before execution; VpnMediator separates payment/access state from reconciliation and recovery; the x86-64 backend checks register-allocation decisions with an independent verifier and differential execution.

Architecture ownership

UniversalToolchain exposes one public LanguageCompiler that resolves composition into an immutable LanguagePlan; runtime materializes the already selected graph.

State & recovery

VpnMediator uses explicit payment/access states, idempotency, reconciliation, migrations, backup/restore, health gates, and rollback.

Correctness boundaries

The x86-64 backend checks allocation with an independent assignment verifier and compares native execution against a reference interpreter oracle.

EXPERIENCE

- 2026 — present

ISP RAS — Static Analysis Engineer

 - Contribute to SharpChecker, ISP RAS's C#.NET static-analysis platform.
 - Current work includes symbolic-computation tasks in C#.NET program analysis.
- July — August 2026

MCST — Compiler Engineering Intern

 - Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.
 - Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.

PROJECTS

UniversalToolchain

One public semantic planner resolves dependencies/conflicts, selects providers, and freezes contribution ordering plus typed backend routes into one immutable LanguagePlan before runtime starts.

The design is enforced by pre-execution validation, exact manifest/ownership checks, and interpreter/CIL parity; the current exact manifest is 1,324/1,324.

VpnMediator

Explicit payment/access states separate external-event acceptance from idempotent reconciliation and recovery; migrations and operational gates keep lifecycle transitions inspectable.

Recovery includes backup/restore, health-gated release switching, and rollback; the public case study excludes secrets and user data.

x86-64 Compiler Backend & Register Allocation

Register-allocation strategy is separated from an independent assignment verifier before phi lowering and SysV x86-64 emission.

Generated native behavior is differentially checked against a reference interpreter; tests cover branches, loops, spills, and parallel phi moves.

SKILLS

Architecture

ownership/module boundaries, typed contracts, immutable planning, extensibility boundaries, explicit state machines, lifecycle/recovery, architecture verification

Compiler / Runtime Platforms

.NET, LanguageCompiler/LanguagePlan, dependency/conflict resolution, provider selection, deterministic pass ordering, typed backend routes

State & Reliability

idempotency, reconciliation, migrations, backup/restore, health gates, rollback, failure containment

Verification

regression tests, exact manifests, independent verifiers, interpreter/backend parity, differential execution

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.

Moscow / remote